AI ENGINEERING COHORT · DELIVERABLE 2 · WEEK 1

Design Opportunities

Memory Admission, Ranking & Conflict Resolution, Lifecycle

Abhijeet Hiwale

July 2026

research/week-1/design_opportunities.pdf

Where This Week's Scan Points

This week covered three components flagged in Deliverable 1 as the least resolved: what counts as worth storing, how conflicting memories get ranked and reconciled, and how memories should change or fade over time. Six ideas came out of the scan as adopt-or-prototype candidates (full matrix in *idea_evaluation_matrix.md*). Three are worth walking through in detail here, because together they change a meaningful piece of the eventual system design rather than just tuning a parameter.

1. Admission Isn't Binary — It's Four Operations

Problem: Deliverable 1 left "what should be stored?" as an open question with no real answer beyond a similarity threshold. A threshold treats admission as a single yes/no decision, which doesn't leave room for updating or retiring something already stored.

What the research shows: Recent memory-management systems (Memory-R1, AgeMem) treat every candidate memory as going through one of four operations — add, update, delete, or do nothing — rather than a single store/discard call. Separately, a 2026 admission-scoring paper (A-MAC) breaks the underlying decision into three independent factors: utility, confidence, and novelty, instead of one blended score. And a widely cited state-of-the-field report from Mem0 flagged that systems which only extract memory from what the user says under-cover the conversation — the assistant's own commitments and recommendations need to be admitted too, or it ends up contradicting decisions it made itself.

Proposed change: Model admission as an explicit four-operation decision (not a threshold), score candidates on utility/confidence/novelty rather than one number, and extract candidates from both user and assistant turns.

Cost / benefit: Low-to-medium integration cost — this changes the shape of the admission logic rather than adding a new subsystem. The benefit is a system that can actually express "this replaces something I already knew" instead of only "store" or "ignore."

Validation plan: Once Deliverable 3's baseline exists, compare admission decisions from a single-threshold version against the four-operation version on the same conversation set, and check whether contradiction and duplication rates drop.

2. Freshness Can't Be Left to the Model

Problem: Open Question 3 asked how the system should resolve two memories that disagree — say, a preference that changed. The obvious answer is "let the model figure out which one is more recent from context." This week's scan turned up direct evidence that this doesn't work.

What the research shows: A 2026 benchmark study built specifically to test this found that every evaluated memory system — including strong long-context baselines — performed far below what you'd expect on a task that's essentially a recency check, even when the model was told explicitly that higher serial numbers meant newer facts. The paper's conclusion is that freshness has to be tracked deterministically outside the model, not inferred in-context. Graphiti/Zep's production approach does exactly this: every memory carries an explicit `valid_until` field, and when a new fact supersedes an old one, the old one gets closed out with a timestamp instead of just sitting at a lower rank.

Proposed change: Give every memory an explicit validity window. When a new fact is written that supersedes an existing one, close the old record out structurally rather than relying on similarity-plus-recency scoring to quietly prefer the newer one.

Cost / benefit: Medium-to-high cost — detecting that two facts describe the same "slot" (e.g., the same preference) is its own non-trivial problem. The benefit is large: this is the single biggest gap between naive retrieval and something trustworthy enough to act on.

Validation plan: Build a small test set of superseded-preference conversations (mirrors the personal examples from Deliverable 1's failure evidence) and check whether the current-vs-stale fact is retrieved correctly, before and after adding explicit supersession.

3. Decay and Deletion Are Not the Same Decision

Problem: Open Question 2 asked how memories should evolve over time, and Deliverable 1's privacy constraint separately requires that deletion actually work. Treating "this memory matters less now" and "this memory must be gone" as the same mechanism risks getting both wrong — either nothing ever really gets deleted, or useful history gets destroyed just because it went quiet for a while.

What the research shows: A recurring framing across this week's sources splits memory consolidation into four independent levers — importance, merge, decay, and eviction — and treats forgetting itself as reduced accessibility rather than automatic destruction. Deletion, by contrast, shows up as its own category, triggered by an explicit user request or a safety/privacy rule, not by ordinary staleness.

Proposed change: Implement decay as a ranking behavior (lower priority over time, not removal) and deletion as a separate, explicitly triggered path. Adopt the four-lever framing as the organizing structure for the lifecycle component generally.

Cost / benefit: Low organizational cost now — this is mostly a design decision, with actual decay/eviction policies to be picked later. The benefit is avoiding a conflation that would otherwise undermine the deletion guarantee required for privacy.

Validation plan: No experiment needed yet — this gets folded into Deliverable 4's system design directly, and tested properly against Deliverable 6's acceptance check that deletion actually removes a memory from all paths.

> Full disposition of all seven ideas scanned this week — including the two marked "prototype" rather than "adopt" — is in *idea_evaluation_matrix.md*. Challenge notes from the design review will be logged in *challenge_notes.md* after the live session.